TESTING OF SOFTWARE SYSTEM

اختبار انظمة البرمجيات

SWE 5325

TOPIC 4

2021

Outline

- What is a Test Case and How to Write Test Cases?
- Why do we Write Tests?
- How to Write Test Cases?
- How to Improve Test Case Efficiency
- Reasons to Reuse Test Cases
- What is a Standard Test in Web Testing?

What is a Test Case and How to Write Test Cases?

- Writing effective cases is a skill. You can learn it from the experience and knowledge of the application under test.

Levels of Test writing process:

- **Level 1:** In this level, you will write the **basic cases from the available specification** and user documentation.
- **Level 2:** This is the **practical stage** in which writing cases depend on the actual functional and system flow of the application.
- **Level 3:** This is the stage in which you will group some cases and **write a test procedure**. The test procedure is nothing but a group of small cases.
- **Level 4: Automation of the project.** This will minimize human interaction with the system and thus the QA can focus on the currently updated functionalities to test rather than remaining busy with Regression testing.

Why do we Write Tests?

- Spot ambiguities early
- Write better code in the first place
- Better design
- Easier debugging
- Up to Date Code Documentation
- Measure Progress
- Catch Regression Early
- Refactor with confidence
- Save time and enjoy!
- Release with confidence

How to Write Test Cases?

- **Important Factors Involved in Writing Process:**

a) TCs are prone to regular revision and update:

b) b) TCs are prone to distribution among the testers who will execute these

c) TCs are prone to Clustering and Batching

d) TCs have a tendency of inter-dependence

e) TCs are prone to distribution among the developers (especially in Test-driven development environment)

Tips to write effective TCs.

- **Keep it simple but not too simple; make it complex, but not too complex**

This statement seems a paradox. But, we promise it is not so. Keep all the steps of TCs atomic and precise. Mention the steps with the correct sequence and correct mapping to the expected results. The test case should be self-explanatory and easy to understand. This is what we mean to make it simple.Making it complex means to make it integrated with the Test Plan and other TCs. Refer to the other TCs, relevant artifacts, GUIs, etc. where and when required. But, do this in a balanced way. Do not make a tester move back and forth in the pile of documents for completing a single test scenario. Do not even let the tester to document these TCs compactly. While writing TCs, always remember that you or someone else will have to revise and update these.

Tips to write effective TCs.

- **After documenting the Test cases, review once as Tester**

Never think that the job is done once you have written the last TC of the test scenario. Go to the start and review all the TCs once, but not with the mindset of a TC writer or Testing Planner. Review all TCs with the mind of a tester. Think rationally and try to dry run your TCs.

Evaluate all the Steps and see if you have mentioned these clearly in an understandable way and the expected results are in harmony with those steps.

Ensure that the test data specified in TCs is feasible not only for actual testers but is according to the real-time environment too. Ensure that there is no dependency conflict among TCs and verify that all the references to other TCs/artifacts/GUIs are accurate. Otherwise, the Testers may be in great trouble.

Tips to write effective TCs.

- **Bound as well as ease the Testers**

Do not leave the test data on testers. Give them a range of inputs especially where calculations are to be performed or the application's behavior depends on inputs. You can let them decide the test data item values but never give them the liberty to choose the test data items themselves.

Because, intentionally or unintentionally, they may use the same test data again & again and some important test data may be ignored during the execution of TCs.

Tips to write effective TCs.

- **Be a Contributor**

Never accept the Design Document as it is. Your job is not just to go through the Document and identify the Test Scenarios. Being a QA resource, never hesitate to contribute to business and give suggestions if you feel that something can be improved in the application.

Suggest to developers too, especially in TC-driven development environment. Suggest the drop-down lists, calendar controls, selection-list, group radio buttons, more meaningful messages, cautions, prompts, improvements related to usability, etc.

Being a QA, don’t just test but make a difference!

Tips to write effective TCs.

- **Never Forget the End User**

The most important stakeholder is the ‘End User’ who will finally use the application. So, never forget him at any stage of TC’s writing. In fact, the End User should not be ignored at any stage throughout the SDLC. Yet, our emphasis so far is just related to the topic.

So, during the identification of test scenarios, never overlook those cases which will be mostly used by the user or the cases which are business-critical even if they are less frequently used. Keep yourself in the shoes of the End User and then go through all the TCs and judge the practical value of executing all your documented TCs.

Reasons to Reuse Test Cases

1) Most functional areas of a website are almost-login, registration, add to cart, wish list, checkout, shipping options, payment options, product page content, recently viewed, relevant products, promo code facilities, etc.

2) Most of the projects are just enhancements or changes to the existing functionality.

3) Content management systems that define the slots for image uploads with static and dynamic ways are also common for all websites.

Reasons to Reuse Test Cases,Cont…

4) Retail websites have **CSR** (Customer Service) system too.

5) Backend system and warehouse application using JDA are also used by all websites.

6) The concept of cookies, timeout, and security are common too.

7) Web-based projects are frequently prone to requirement changes.

8) The types of testing needed are common, like browser compatibility testing, performance testing, security testing

What is a Standard Test in Web Testing?

- Create test cases that are complete – steps, data, variables, etc. This will ensure that the non-similar data/variable can simply be replaced when a similar test case is required.
- The entrance and exit criteria should be properly defined.
- The modifiable steps or the statement in the steps should be highlighted in a different color for quick find and replace.
- The language used for the standard test case creation should be generic.

What is a Standard Test in Web Testing?

- All the features of each website should be covered in the test cases.
- The name of the test cases should be the name of the functionality or the feature that the test case is covering.
- If there is any basic or standard sample or GUI file or screenshot of the feature, then it should be attached with the relevant steps.

Test Case template

Test Scenario ID		Test Case ID	
Test Case Description		Test Priority	
Pre-Requisite		Post-Requisite	

Test Execution Steps:

S.No	Action	Inputs	Expected Output	Actual Output	Test Browser	Test Result	Test Comments

Test Case Example

Test Scenario ID	Login-1	Test Case ID	Login-1B
Test Case Description	Login – Negative test case	Test Priority	High
Pre-Requisite	NA	Post-Requisite	NA

Test Execution Steps:

S.No	Action	Inputs	Expected Output	Actual Output	Test Browser	Test Result	Test Comments
1	Launch application	https://www.facebook.com/	Facebook home	Facebook home	IE-11	Pass	[Priya 10/17/2017 11:44 AM]: Launch successful
2	Enter invalid Email & any Password and hit login button	Email id : invalid@xyz.com Password: ******	The email address or phone number that you've entered doesn't match any account. Sign up for an account.	The email address or phone number that you've entered doesn't match any account. Sign up for an account.	IE-11	Pass	[Priya 10/17/2017 11:45 AM]: Invalid login attempt stopped
3	Enter valid Email & incorrect Password and hit login button	Email id : valid@xyz.com Password: ******	The password that you've entered is incorrect. Forgotten password?	The password that you've entered is incorrect. Forgotten password	IE-11	Pass	[Priya 10/17/2017 11:46 AM]: Invalid login attempt

How to write Test Cases for a Login Page

- We usually write test cases for the login page for every application we test. Every login page should have the following elements.
- ‘**Email/Phone Number/Username**‘ Textbox
- ‘**Password**‘ Textbox
- ‘**Login**‘ Button
- ‘**Remember Me**‘ Checkbox
- ‘**Keep Me Signed In**‘ Checkbox
- ‘**Forgot Password**‘ Link
- ‘**Sign up/Create an account**‘ Link
- **CAPTCHA**

Focus on what ?

- we focus on the following to write test cases for a login page:
- We have to write test cases on each object in the login form.
- We have to write both positive and negative test cases.
- We have to write both funcitonal and nonfunctional test cases.
- We have to write UI, functional, compatibility and performance test cases.

- The list consists of both Positive and Negative test scenarios of login page along with UI test cases, Security test cases and so on.

UI Test Scenarios of Login Page

- Verify that the login screen contains elements such as Username, Password, Sign in button, Remember password check box, Forgot password link, and create an account link.
- Verify that all the fields such as Username, Password has a valid placeholder
- Verify whether all the text boxes have a minimum and maximum length.
- Verify that the labels float upward when the text field is in focus or filled (In case of the floating label)
- Verify to see if the font style and size of the labels, as well as the text on each object, are clearly visible.
- Verify that the application's user interface (UI) is responsive, so it will adapt to different screen resolutions and devices.
- Verify the login page and all the fields in the login page are displaying without any break in different browsers

Functional Test Scenarios of Login Page

- Verify that cursor is focused on the “Username” text box on the page load (login page)
- Verify that tab functionality is working properly or not
- Verify that Enter/Tab key works as a substitute for the Sign-in button
- Verify that the User is able to Login with Valid Credentials
- Verify that the User is not able to Login with an invalid Username and invalid Password
- Verify that the User is not able to Login with a Valid Username and invalid Password
- Verify that the User is not able to log in with an invalid Username and Valid Password

Functional Test Scenarios of Login Page

- Verify that clicking on the browser back button after successful login should not take the User to log out mode
- Verify that validation message is displayed in the case when User leaves Username or Password as blank
- Verify that validation message is displayed in case of exceeding the character limit of the Username and Password fields
- Verify that validation message is displayed in case of entering special character in the Username and password fields
- Verify that the “Keep me logged in” checkbox is unselected by default (depends on business logic, it may be selected or unselected)
- Verify that the timeout of the login session (Session Timeout)

Functional Test Scenarios of Login Page

- Verify that the logout link is redirected to login/home page
- Verify that User is redirected to appropriate page after successful login
- Verify that the User is redirected to the Forgot password page when clicking on the Forgot Password link
- Verify that the User is redirected to the Create an account page when clicking on the Signup / Create an account link
- Verify that the User should be able to login with the new password after changing the password
- Verify that the user should not be able to login with the old password after changing the password

Functional Test Scenarios of Login Page

- Verify that spaces should not be allowed before any password characters attempted
- Verify whether the user is still logged in after a series of actions such as sign-in, close the browser, and reopen the application.
- Verify that the ways to retrieve the password if the user forgets the password

Non-functional Security Test Cases for Login Page

- Verify that clicking on the browser back button after successful logout should not take the User to a logged-in mode
- Verify that there is a limit on the total number of unsuccessful login attempts (No. of invalid attempts should be based on business logic. Based on the business logic, User will be asked to enter the captcha and try again or user will be blocked)
- Verify that the password is in encrypted form (masked format) when entered in the password field.
- Verify the password can be copy-pasted. System shouldn't allow users to copy paste password.
- Verify that encrypted characters in the "Password" field should not allow deciphering if copied

Non-functional Security Test Cases for Login Page

- Verify that the “Remember password” checkbox is unselected by default (depends on business logic, it may be selected or unselected)
- Verify whether the login form is revealing any security information by viewing the page source
- Verify that the login page is vulnerable to SQL injection.
- Verify whether Cross-site scripting (XSS) vulnerability works on a login page. XSS vulnerability may be used by hackers to bypass access controls.

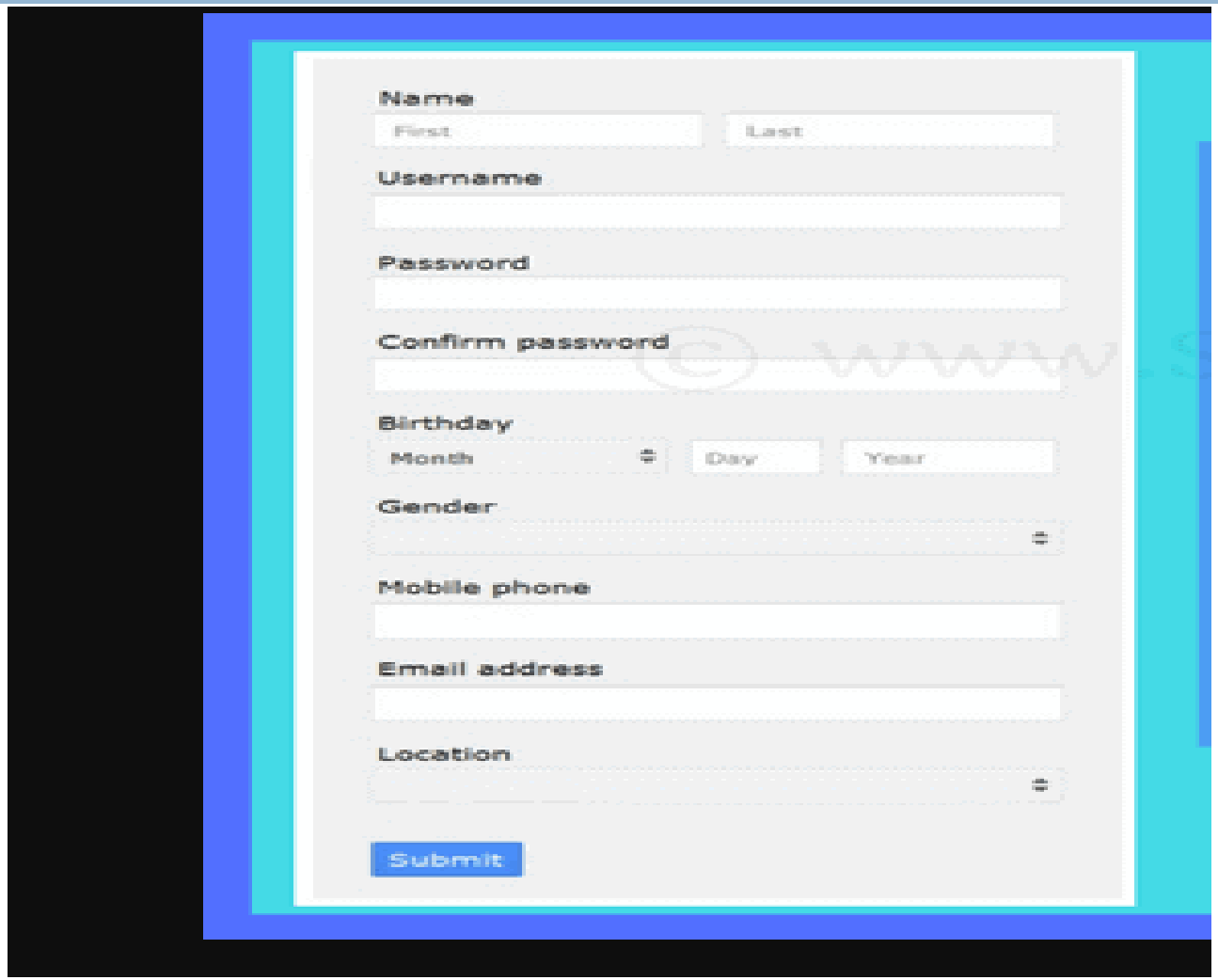
Test Cases for CAPTCHA & Cookies (If there is a captcha on the login page)

- Verify that whether there is a client-side validation when the User doesn't enter the CAPTCHA
- Verify that the refresh link of CAPTCHA is generating the new CAPTCHA
- Verify that the CAPTCHA is case sensitive
- Verify whether the CAPTCHA has audio support to listen
- Verify whether virtual keyboard is available and working properly to enter login credentials incase of banking applications.
- Verify two-way authentication through OTP is working properly incase of banking applications.
- Verify SSL certificate is implemented or not

Test Cases for CAPTCHA & Cookies (If there is a captcha on the login page)

- Verify that the user is able to login when the browser cookies are cleared. When the cookies are cleared, system should not allow user to login automatically.
- Verify the login functionality when the browser cookies are turned off.

Test Scenarios Registration Form [Write Test Cases of Signup Form]



Test Scenarios of a Registration Form:

- Verify that the Registration form contains Username, First Name, Last Name, Password, Confirm Password, Email Id, Phone number, Date of birth, Gender, Location, Terms of use, Submit, Login (If you already have an account)
- Verify that tab functionality is working properly or not
- Verify that Enter/Tab key works as a substitute for the Submit button
- Verify that all the fields such as Username, First Name, Last Name, Password and other fields have a valid placeholder
- Verify that the labels float upward when the text field is in focus or filled (In case of floating label)
- Verify that all the required/mandatory fields are marked with * against the field

Test Scenarios of a Registration Form:

- Verify that system generates a validation message when clicking on submit button without filling all the mandatory fields.
- Verify that entering blank spaces on mandatory fields lead to validation error
- Verify that clicking on submit button by leaving optional fields, submits the data to the server without any validation error
- Verify that case sensitivity of Username (usually Username field should not follow case sensitivity – ‘rajkumar’ & ‘RAJKUMAR’ acts same)
- Verify that system generates a validation message when entering existing username
- Verify that the character limit in all the fields (mainly username and password) based on business requirement
- Verify that the username validation as per business requirement (in some application, username should not allow numeric and special characters)

Test Scenarios of a Registration Form:

- Verify that the validation of all the fields are as per business requirement
- Verify that the date of birth field should not allow the dates greater than current date (some applications have age limit of 18 in that case you have to validate whether the age is greater than or equal to 18 or not)
- Verify that the validation of email field by entering incorrect email id
- Verify that the validation of numeric fields by entering alphabets and characters
- Verify that leading and trailing spaces are trimmed after clicking on submit button
- Verify that the “terms and conditions” checkbox is unselected by default (depends on business logic, it may be selected or unselected)
- Verify that the validation message is displayed when clicking on submit button without selecting “terms and conditions” checkbox
- Verify that the password is in encrypted form when entered

TESTING OF SOFTWARE SYSTEM

اختبار انظمة البرمجيات

SWE 5325

TOPIC 5

2021

Outline

- Test Planning
- Benefit of test strategy
- Example of test strategy
- Test Strategy

Test Plan

- A Test Plan is defined as a document which outlines the scope, objective, method and weight on a software testing task

Test plan---Introduction

- **Introduction**

Brief introduction of the test strategies, process, workflow and methodologies used for the project

- **Scope**
  - **In Scope**
    Scope defines the features, functional or non-functional requirements of the software that **will be** tested
  - **Out of Scope**

    Out Of Scope defines the features, functional or non-functional requirements of the software that **will NOT be** tested

Test plan-(Introduction)--**Quality Objective**

make a mention of the overall objective that you plan to achieve without your testing Some objectives of your testing project could be

- Ensure the Application Under Test conforms to functional and non-functional requirements
- Ensure the AUT meets the quality specifications defined by the client
- Bugs/issues are identified and fixed before go live

Test plan-(Introduction)--**Roles and Responsibilities**

Detail description of the Roles and responsibilities of different team members like

- QA Analyst
- Test Manager
- Configuration Manager
- Developers
- Installation Team Amongst others

Test plan-(Test Methodology)-- Test Methodology

Mention the reason of adopting a particular test methodology for the project. The test methodology selected for the project could be

- WaterFall
- Iterative
- Agile
- Extreme Programming

Test plan-(**Test Methodology**)-- **Test Levels**

- **Test Levels define the Types of Testing to be executed on the Application Under Test (AUT)**. The Testing Levels primarily depends on the scope of the project, time and budget constraints.

Test plan-(**Test Methodology**)-- **Bug Triage**

Bug triage is a process where tracker issues are screened and prioritised. Triage should help ensure we appropriately manage all reported issues - bugs as well as improvements and feature requests.

The goal of the triage is to

- To define the type of resolution for each bug
- To prioritize bugs and determine a schedule for all “To Be Fixed Bugs’.

Test plan-(**Test Methodology**)-- **Test Completeness**

define the criterias that will deem your testing complete.

For instance, a few criteria to check Test Completeness would be

- 100% test coverage
- All Manual & Automated Test cases executed
- All open bugs are fixed or will be fixed in next release

Test plan-(**Test Deliverables**)

- Test Plan
- Test Cases
- Requirement Traceability Matrix
- Bug Reports
- Test Strategy
- Test Metrics
- Customer Sign Off

Test plan-(**Resource & Environment Needs**)

Testing Tools

- Make a list of Tools like
- Requirements Tracking Tool
- Bug Tracking Tool
- Automation Tools

Test Environment

It mentions the minimum **hardware** requirements that will be used to test the Application.

Following **software's** are required in addition to client-specific software.

- Windows 8 and above
- Office 2013 and above
- MS Exchange, etc.

Test plan-(**Terms/Acronyms**)

- **Terms/Acronyms**

Term	Definition
API	Application Program Interface
AUT	Application Program Interface

Test Strategy

- **Test Strategy** in software testing is defined as a set of guiding principles that determines the test design & regulates how the software testing process will be done. The objective of the Test Strategy is to provide a systematic approach to the software testing process in order to ensure the quality, traceability, reliability and better planning.

TESTING OF SOFTWARE SYSTEM

اختبار انظمة البرمجيات

SWE 5325

TOPIC 6

2021

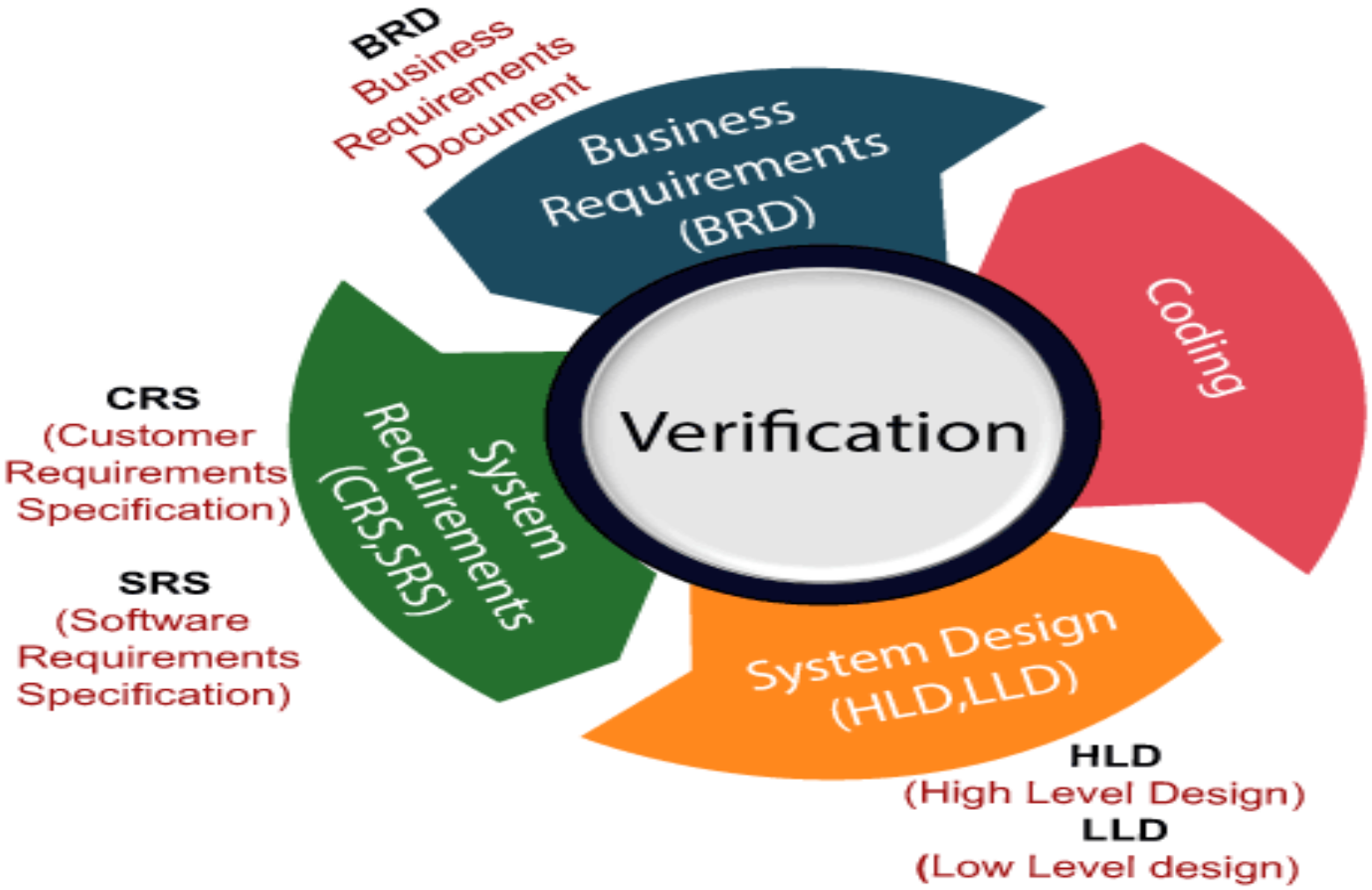
Outline

- Verification and Validation Testing
- Type of testing

Verification testing

- Verification testing includes different activities such as business requirements, system requirements, design review, and code walkthrough while developing a product.
- It is also known as static testing, where we are ensuring that "**we are developing the right product or not**". And it also checks that the developed application fulfilling all the requirements given by the client.

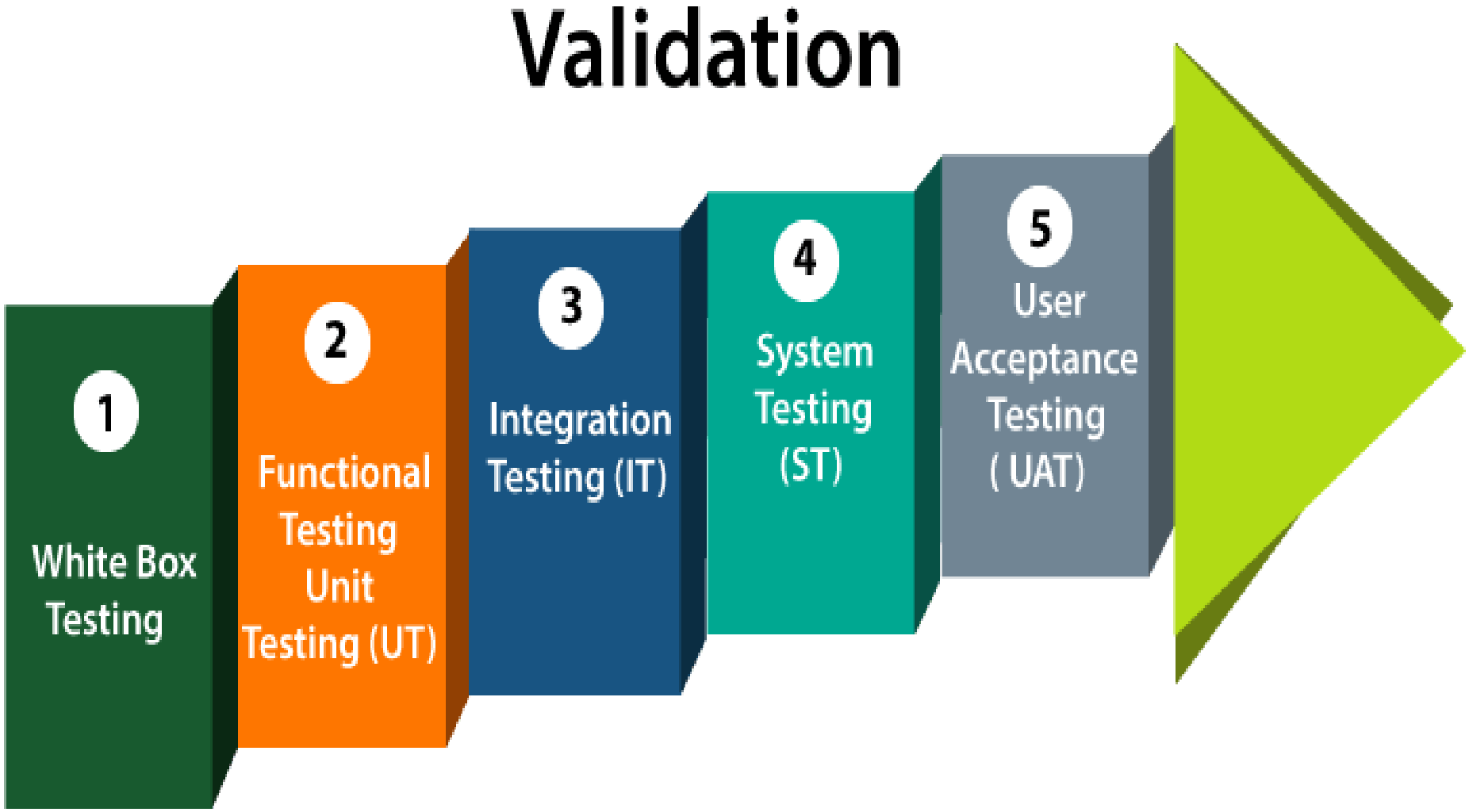
Verification testing



Validation testing

- alidation testing is testing where tester performed functional and non-functional testing. Here functional testing includes Unit Testing (UT), Integration Testing (IT) and System Testing (ST), and non-functional testing includes User acceptance testing (UAT).
- Validation testing is also known as dynamic testing, where we are ensuring that **"we have developed the product right."** And it also checks that the software meets the business needs of the client.

Validation testing



Difference between verification and validation testing(Verification)

- We check whether we are developing the right product or not.
- Verification is also known as **static testing**.
- Verification includes different methods like Inspections, Reviews, and Walkthroughs.
- **Quality assurance** comes under verification testing.
- The execution of code does not happen in the verification testing.
- In verification testing, we can find the bugs early in the development phase of the product.
- Verification is done before the validation testing

Difference between verification and validation testing(Validation)

- We check whether the developed product is right.
- Verification is also known as **daynamic testing**.
- Validation includes testing like functional testing, system testing, integration, and User acceptance testing..
- **Quality control** comes under validation testing.
- In validation testing, the execution of code happens.
- In the validation testing, we can find those bugs, which are not caught in the verification process.
- After verification testing, validation testing takes place.

Type of Testing

- Acceptance testing

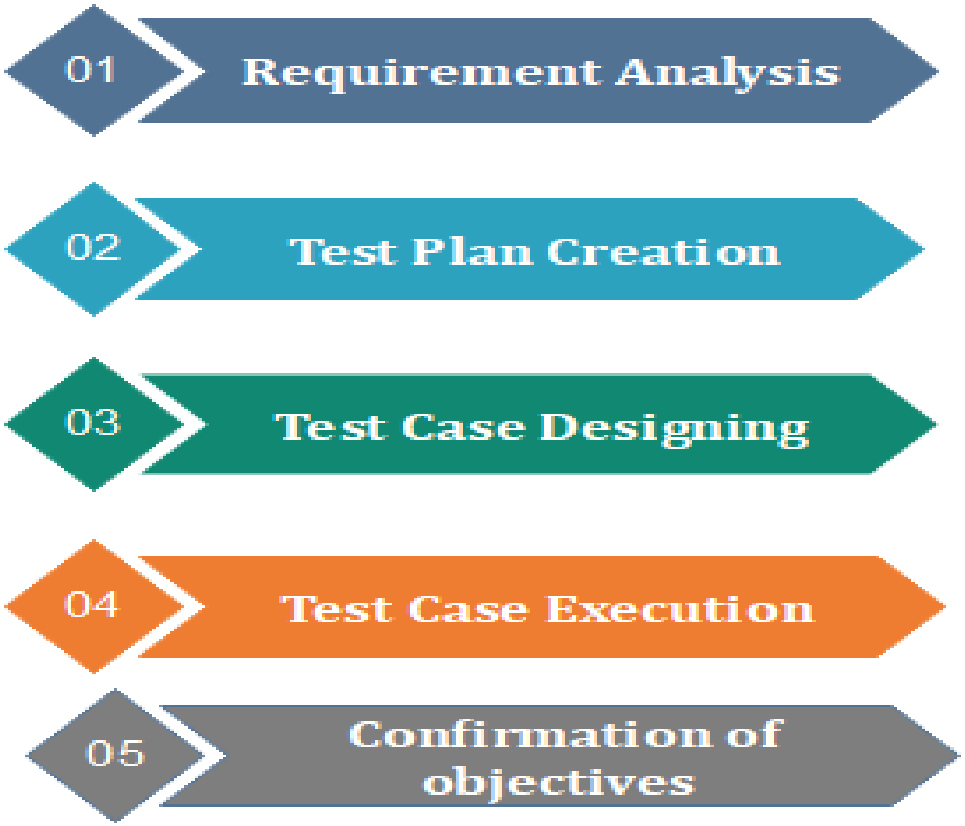
Acceptance testing is formal testing based on user requirements and function processing. It determines whether the software is conforming specified requirements and user requirements or not. It is conducted as a kind of Black Box testing where the number of required users involved testing the acceptance level of the system. It is the fourth and last level of software testing.

Type of Testing(Accepting test) Cont…

- User acceptance testing (UAT) is a type of testing, which is done by the customer before accepting the final product. Generally, UAT is done by the customer (domain expert) for their satisfaction, and check whether the application is working according to given business scenarios, real-time scenarios.

Type of Testing(Accepting test) Cont…

- Steps to Perform Acceptance Testing



Type of Testing(Accepting test) Cont…

Advantages of Acceptance Testing

- It increases the satisfaction of clients as they test application itself.
- The quality criteria of the software is defined in an early phase so that the tester has already decided the testing points. It gives a clear view to testing strategy.
- The information gathered through acceptance testing used by stakeholders to better understand the requirements of the targeted audience.
- It improves requirement definition as client tests requirement definition according to his needs.

Type of Testing(Accepting test) Cont…

Disadvantages of Acceptance Testing

- Customers are not willing to do that; it defeats the whole point of acceptance testing.
- If test cases are written by someone else, the customer does not understand them, so tester has to perform the inspections by themselves only.

Type of Testing(Adhoc Testing)

- we do when the build is in the checked sequence, then we go for Adhoc testing by checking the application randomly.
- Adhoc testing is also known as **Monkey testing and Gorilla testing**.

Type of Testing(Adhoc Testing)



Type of Testing(Adhoc Testing)

When we perform adhoc testing

- We go for Adhoc testing when all types of testing are performed. If the time permits then we will check all the negative scenarios during adhoc testing.

Type of Adhoc Testing

- Following are the types of adhoc testing:
- **Buddy testing**
- **Pair testing**

Type of Testing(Adhoc Testing)

- **Buddy testing**
  Buddy testing is done with at least two members. And one member is from the testing team, and another one is from the development team.
- When the unit testing is performed on the application, then only we can perform buddy testing. This type of testing helps the developer team and testing team to do their jobs.

Type of Testing(Adhoc Testing)

Pair testing

- In this type of testing, two testers will work together to test the software, where they can share their ideas and identify the bugs or defects in the application.
- One of them will test the application, and the other one can review and analyze the application.

Type of Testing(Adhoc Testing)

Advantages of Adhoc Testing

- Adhoc testing cannot follow any process; that why it can be performed anytime in the software development life cycle.
- The test engineer can test the application in their new ways that helps us to find out many numbers of bugs as compared to the actual testing process.
- The developer can also execute adhoc testing while developing the module that helps them to code in a better way.
- When the in-depth testing is required in less time, adhoc testing can be performed and also deliver the quality product on time.
- Adhoc testing does not require any documentation; that's why tester can test the application with more concentration without worrying about the formal documentation.

Type of Testing(Adhoc Testing)

Disadvantages of Adhoc Testing

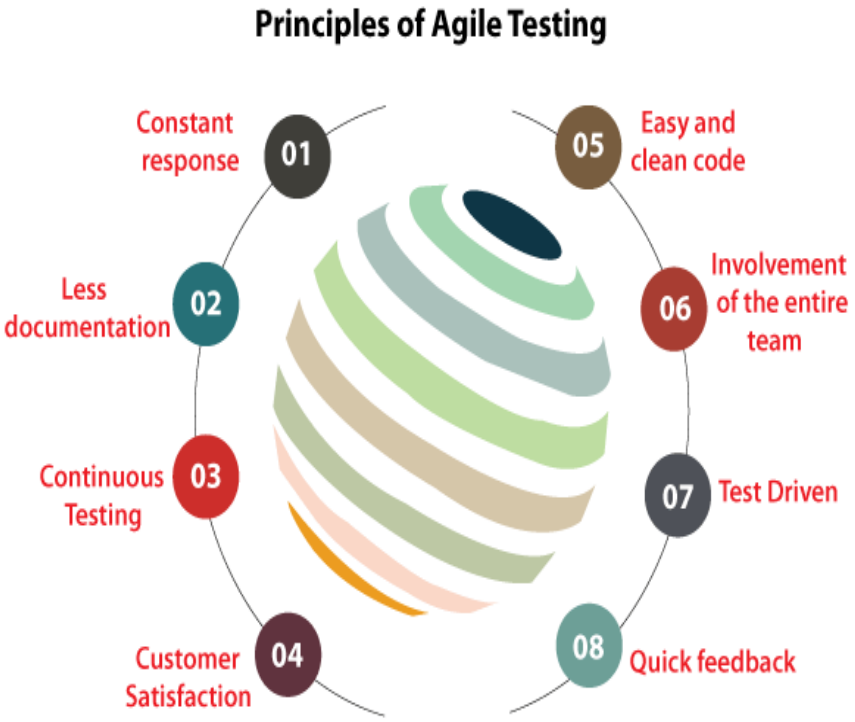
- Adhoc testing is dependent on the test engineer's product knowledge because he/she knows the flow of the application, so he/she knows where the application can collapse, and a new test engineer may not that much familiar with the application.
- Sometimes reproducing the bug is difficult because, in this testing, we did not follow any planning.

Agile Testing

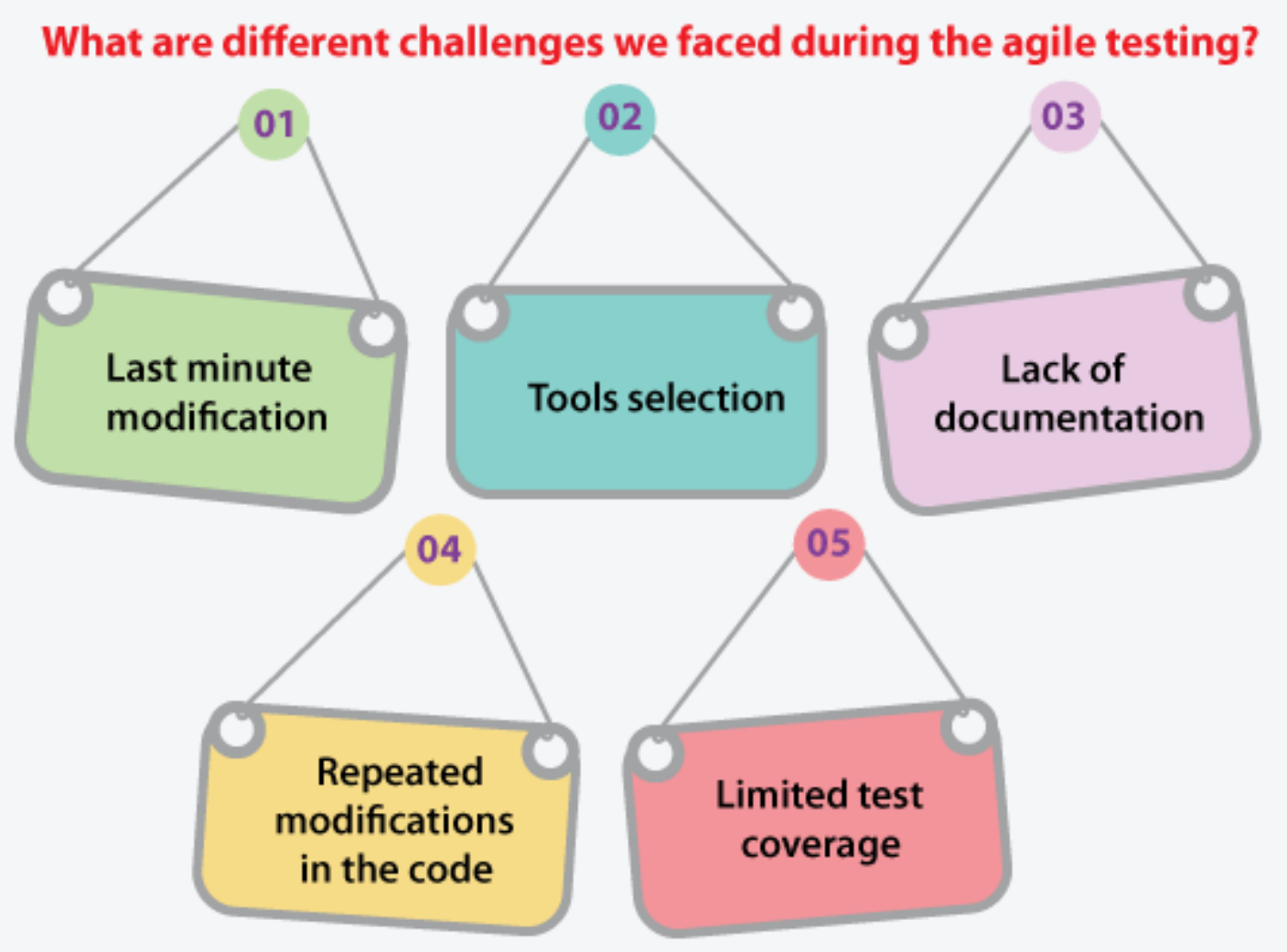
- Agile testing is an **iterative and incremental method,** and the necessities, which develop during the cooperation between the **customer** and **self-establish teams**.

Principles of Agile Testing

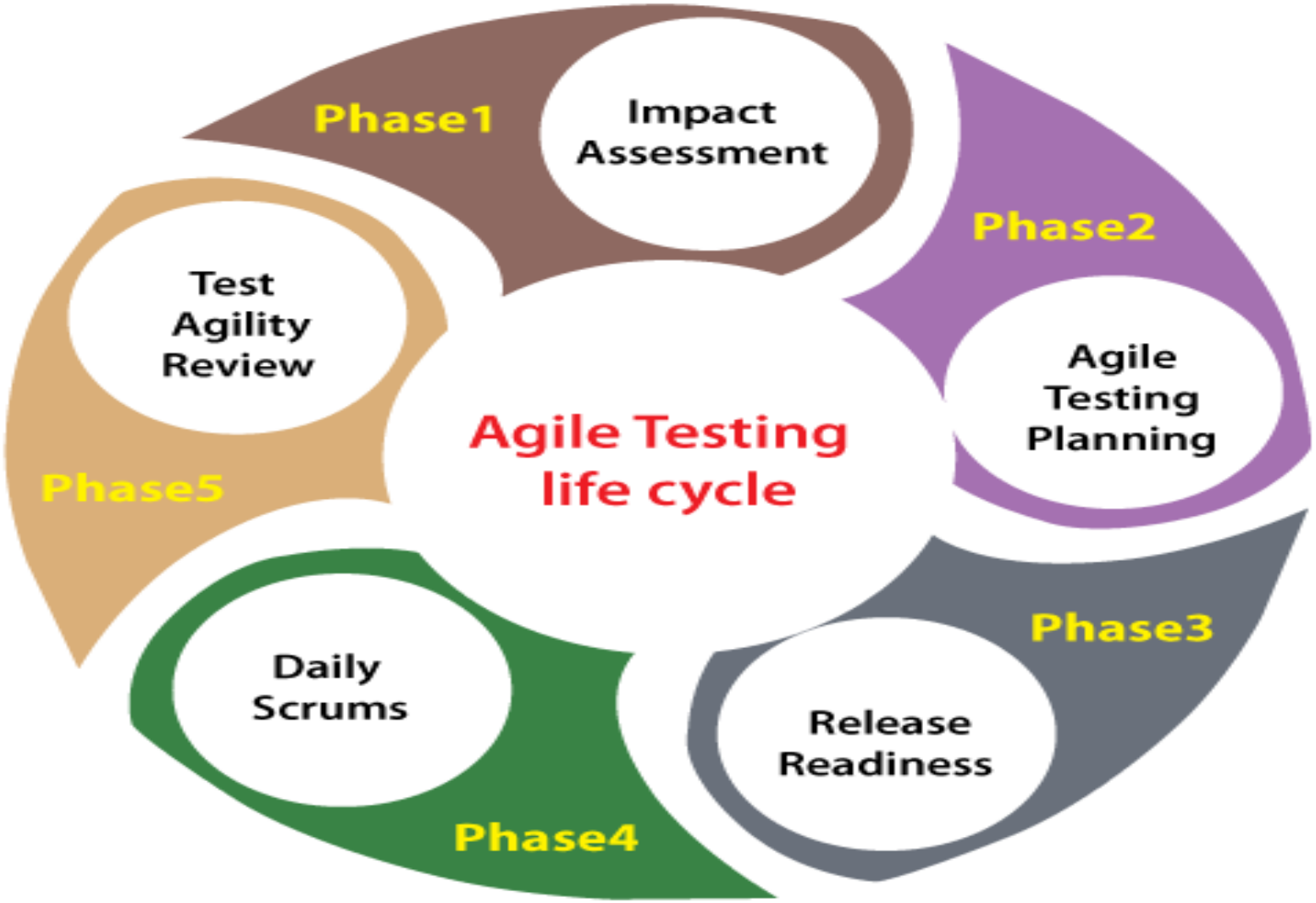
- Constant response
- Less documentation
- Continuous Testing
- Customer Satisfaction
- Easy and clean code
- Involvement of the entire team
- Test-Driven
- Quick feedback



What are the different challenges we faced during the agile testing?



Agile Testing life cycle



Advantages of Agile Testing

- Gives way to get regular **feedback** and reviews directly from the end-user, which helps us enhance the software product's quality and attribute.
- save lots of **time** and **money,** making the estimation of cost more transparent.
- Through daily meetings, we can determine better issues.
- Reduces **documentation**, or we can say it requires less documentation to execute the agile testing.
- Reducing errors and enhancing software productivity.
- The workload is divided into small parts in agile software development and restricts the developer from going off the track. And in the results, we will get more minor inconsistencies and higher efficiency.

Disadvantage of Agile Testing

- That if two or more members leave a job, it will lead to project failure.
- Determination is needed while performing any testing to test the application or the software and becomes inconsistent for the testing team.
- It makes it difficult for us to predict expected results because there are no or fewer documentation results into explicit conditions and requirements.
- Sometimes, it leads us to introduce new bugs in the system because the bug fixes, modifications, and releases repeatedly happen in agile testing.